

Abstract

This project developed a self-hosted Payroll Management System tailored to operational realities in Belizean organizations, where payroll processing is often fragmented across manual records and spreadsheets. The final system was implemented with Rust (`axum`) and PostgreSQL, with a focus on correctness, auditability, and maintainability. Core delivered capabilities include authenticated access control, payroll period lifecycle processing (generate, approve/lock, execute), audit event tracking, and multi-format reporting (PDF/CSV).

The development process followed an iterative progression from architecture definition to backend workflow implementation, followed by frontend integration and reporting features. One of the strongest outcomes was the depth of transaction-safe payroll logic: row-level locks, state transition validation, and constrained schema rules were used to protect financial data integrity. The use of Decimal arithmetic in payroll computations also improved confidence in financial correctness compared with floating-point alternatives.

A key insight from the project is that payroll systems should prioritize data and workflow integrity first, because usability and reporting quality become more reliable when the underlying domain model is strict and explicit. Another insight is that technical quality alone is not sufficient for academic and practical completeness; architecture visualization, usability narrative, and reflective analysis are equally important for demonstrating full project maturity.

Several trade-offs emerged during development. Choosing Rust provided performance and safety advantages but introduced a steeper learning and delivery curve for rapid UI iteration compared with higher-level frameworks. Choosing self-hosting improved data control and local adaptability, but shifted operational responsibility (TLS, backup, monitoring, credential policy) to the deploying institution.

If this project were repeated, the following changes would be made earlier in the timeline: first, include architecture diagrams and evaluation structure from the beginning rather than final-phase consolidation; second, formalize usability feedback sessions with representative users; third, prioritize production hardening tasks such as TLS, CSRF protection, and credential lifecycle controls as core milestones rather than end-stage enhancements.

Overall, the project achieved its primary objectives and produced a technically credible payroll platform suitable for pilot deployment. The final phase strengthened completeness through reflection, visual documentation, and objective-based evaluation, while also identifying a clear roadmap for production readiness and long-term sustainability.